UNIVERSITY OF HERTFORDSHIRE

School of Physics, Engineering and Computer Science

MSc Software Engineering

M7COM1038 Software Engineering Master's Project

27-04-2026

Graph-Based Failure Impact Estimation in Microservice Architectures

Name: Pavithran Gurusamy

Student ID: 24067994

Supervisor: Kelechi Emerole

I confirm that this report has been critically proof-read and quality checked to ensure it is free from grammar, spelling, and formatting errors, and meets appropriate standards of clarity, coherence, and presentation.

I would like to express my sincere gratitude to my supervisor, Kelechi Emerole, for his guidance, constructive feedback, and support throughout this project. His advice and discussions were especially valuable in helping me refine ideas and overcome challenges during the research process.

MSc FPR Declaration

This report is submitted in partial fulfilment of the requirements for the degree of: Master of Science in Software Engineering at the University of Hertfordshire (UH).

I hereby declare that the work presented in this project and report is entirely my own, except where explicitly stated otherwise. All sources of information and ideas, whether quoted directly or paraphrased, have been properly referenced in accordance with academic standards. I understand that any failure to properly acknowledge the work of others could constitute plagiarism and may result in academic penalties.

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on the university website provided the source is acknowledged.

Table of Contents

Table of Figures

Table of Tables

1 Abstract

Microservice architectures are widely used in cloud systems because they support scalability, modularity, and independent deployment. However, their distributed nature also increases reliability challenges, as services often depend on multiple connected components. A failure in one service can therefore propagate through dependency paths and cause wider disruption across the system. Identifying critical services and estimating system vulnerability is an important challenge for improving resilience in cloud-native architectures.

This project proposes a graph-based analytical model for assessing cascading failure impact in microservice systems. The architecture is represented as a directed dependency graph, where services are modelled as nodes and dependencies as weighted edges. Reliability parameters, including failure rate, propagation probability, and Mean Time to Recovery (MTTR), are incorporated to estimate the expected disruption caused by service failures. A Java-based tool was developed and evaluated using multiple scenarios, including baseline, low propagation, high propagation, high MTTR, and high failure rate conditions.

The model generated service-level impact scores and a System Vulnerability Index (SVI) to compare overall system risk across scenarios. The results showed that vulnerability increased under higher failure-rate and recovery-time conditions, while reduced propagation probability lowered overall system exposure. These findings indicate that both dependency structure and recovery behaviour have a significant influence on cascading failure impact.

The study demonstrates that graph-based analytical modelling can provide a practical and interpretable method for identifying high-risk services and supporting reliability planning in cloud microservice architectures.

2 Introduction

2.1 Problem Overview

Modern organisations increasingly depend on cloud-based digital services that must remain continuously available, scalable, secure, and responsive. To meet these operational demands, many enterprises have adopted microservice architectures, where large applications are decomposed into smaller, independently deployable services that communicate through APIs, messaging systems, and shared cloud infrastructure. This architectural style offers important benefits such as rapid deployment, independent scalability, technological flexibility, and easier maintenance of individual components. As a result, microservices have become a common design model for large-scale cloud-native systems (Velepucha and Flores, 2023).

Despite these advantages, microservice environments introduce significant reliability challenges. Unlike monolithic systems, where most functionality is contained within one application boundary, microservice systems depend on many interconnected components operating across distributed infrastructure. These components may include application services, authentication layers, databases, queues, caching services, storage platforms, load balancers, and external APIs. Because of these dependencies, failure in one component can affect several others, allowing disruption to spread through the wider system. In practice, a relatively minor fault may escalate into a cascading failure, reducing performance or causing a partial or total service outage. Dependency concentration, excessive coupling, and over-centralised shared services may also reflect recognised microservice architectural bad smell patterns that increase operational fragility (Taibi and Lenarduzzi, 2018).

The significance of this problem extends beyond technical reliability. Service disruption can generate direct financial loss, reputational damage, customer churn, missed service-level agreements, and increased recovery costs. In sectors such as finance, healthcare, retail, transport, and media platforms, even short outages may affect thousands of users and critical business processes. There are also professional and ethical considerations where failures impact access to essential digital services or expose weak operational governance. Consequently, improving resilience in distributed software systems is both a technical and commercial priority.

2.2 Current Issues

Although modern organisations deploy monitoring systems, observability tools, auto-scaling platforms, and recovery automation, many reliability approaches remain reactive. Incidents are often addressed after faults have already occurred. Existing resilience techniques, such as retries, circuit breakers, fallbacks, replication, and graceful degradation, reduce local failure impact, but they do not always indicate which service poses the greatest systemic risk to the wider architecture. Chaos engineering can reveal weaknesses through controlled fault injection, but it may require

mature infrastructure, specialised expertise, and operational tolerance for experimentation (Mailewa et al., 2025).

This creates an important challenge in current practice. Organisations may understand how to recover from failure, but often have limited visibility into how dependency structures amplify disruption before incidents occur. Existing literature discusses microservice design, resilience patterns, and fault tolerance extensively, yet there remains less emphasis on lightweight analytical methods that quantify expected cascading impact across service ecosystems. Many approaches focus on isolated service reliability rather than whole-system vulnerability, while architectural bad smells may remain hidden until failures occur.

2.3 Project Details

This project addresses that gap by developing a graph-based analytical framework for estimating failure impact in microservice architectures. The system is modelled as a directed dependency graph in which services are represented as nodes and relationships between services are represented as edges. If one service depends on another for data, processing, communication, or authentication, a directed dependency is established. This enables the architecture to be analysed structurally and operationally using graph theory principles, similar to failure propagation studies in other complex engineered systems (O'Halloran et al., 2021).

To make the model practically meaningful, operational reliability parameters are incorporated into the graph. These include:

- **Failure Rate (λ)** – likelihood that a service experiences failure
- **Propagation Probability** – likelihood that disruption spreads to dependent services
- **Mean Time to Recovery (MTTR)** – expected duration required to restore a failed service

By combining topology with these metrics, the framework estimates how often a service may fail, how widely disruption may spread, and how severe the resulting downtime may become.

The practical implementation of the project was developed using Java with CSV-based configuration files to ensure reproducibility and flexible scenario testing. A realistic cloud-native service environment was modelled using components aligned with common Google Cloud Platform style deployments, including frontend delivery, APIs, messaging layers, storage, database services, caching, DNS, CDN, and load balancing. This architecture was intentionally selected to reflect the behaviour of real distributed production systems rather than a purely theoretical network.

The project then evaluates multiple operational scenarios, including baseline conditions, reduced propagation probability, increased propagation probability, higher recovery time, and increased failure rates. For each scenario, the model calculates service-level impact scores and system-level indicators such as the System Vulnerability Index (SVI), which represents the aggregate

vulnerability of the architecture. These outputs allow comparison of how changing reliability conditions influence systemic risk.

2.4 Aims and Objectives

The main aim of this project is to determine whether a lightweight analytical model can identify critical services and quantify vulnerability across changing cloud operating conditions. The objectives of the study are:

- To represent a microservice system as a dependency graph
- To incorporate realistic reliability parameters into the model
- To estimate cascading failure impact for each service
- To compare system vulnerability under multiple scenarios
- To identify the highest-risk components requiring resilience prioritisation

2.5 Research Question and Novelty

The central research question guiding this dissertation is:

Can graph-based analytical modelling, using dependency graphs and scenario-based output metrics, provide a practical and interpretable method for quantifying cascading failure impact and system risk in cloud microservice architectures?

The novelty of this research lies in combining dependency graph analysis with operational reliability metrics and scenario-based evaluation within a repeatable and computationally efficient framework. While existing studies often examine resilience mechanisms, simulation-intensive testing, or static network analysis separately, this study integrates architectural structure, failure likelihood, recovery behaviour, and measurable impact indicators into a unified decision-support model. This provides practical value for system architects, site reliability engineers, and cloud organisations seeking faster methods of risk assessment during design reviews, migration planning, or resilience investment decisions.

2.6 Feasibility, Commercial Context, and Risk

From a feasibility perspective, the project was achievable within the available time, technical scope, and resource constraints because it uses synthetic but configurable datasets, modular implementation, and controlled scenario analysis rather than requiring access to sensitive enterprise production systems. Commercially, similar methods could support cost reduction by

helping organisations target monitoring, redundancy, and engineering resources more effectively. However, adoption risks include simplified modelling assumptions, variation between real infrastructures, and overreliance on analytical outputs without professional engineering judgement. These limitations are critically examined later in the dissertation.

2.7 Report Structure

The remainder of this report is structured as follows. Chapter 3 reviews literature on microservice architectures, fault tolerance, resilience testing, and graph-based propagation modelling. Chapter 4 explains the research methodology, dependency graph design, datasets, metrics, and implementation approach. Chapter 5 presents scenario-based results and evaluates service-level impact, cascading behaviour, and system-wide vulnerability findings. Chapter 6 discusses implications, limitations, and practical relevance for cloud reliability engineering. Finally, Chapter 7 concludes the study and outlines recommendations for future research.

3 Literature Review

3.1 Introduction to the Field

Cloud computing has transformed the way organisations develop and operate digital systems. Many enterprises have moved from traditional monolithic software architectures toward microservice architectures, where applications are decomposed into smaller, independently deployable services that communicate through APIs, event streams, or messaging systems. This shift supports scalability, continuous delivery, technology flexibility, and faster response to changing business needs (Velepucha and Flores, 2023).

Although microservices provide clear engineering and commercial benefits, they also introduce significant operational complexity. Business processes may depend on long chains of interacting services, databases, caches, queues, authentication layers, and network components. As a result, the failure of a single component can spread through dependency relationships and create wider service disruption. This behaviour is commonly described as a cascading failure and represents an important challenge for cloud-native reliability engineering.

The aim of this dissertation is to investigate whether graph-based analytical modelling can provide a practical and interpretable method for quantifying cascading failure impact and system risk in cloud microservice architectures. This literature review evaluates the principal theories, technologies, and research approaches relevant to that aim, while identifying where current knowledge remains incomplete.

3.2 Key Studies and Works

A substantial body of literature focuses on the architectural benefits of microservices. Velepucha and Flores (2023) argue that microservices improve maintainability, scalability, and deployment agility when compared with monolithic systems. By allowing services to be deployed independently, organisations can reduce release bottlenecks and scale only the components under demand. These advantages help explain the widespread adoption of cloud-native architectures.

However, while such studies strongly support modularity benefits, they often devote less attention to the reliability risks created by runtime dependency chains. In distributed environments, service autonomy does not eliminate coupling entirely; rather, coupling frequently shifts from internal code relationships to network-based operational dependencies. Taibi and Lenarduzzi (2018) note that certain recurring design weaknesses, often described as microservice bad smells, can emerge when systems become excessively coupled, overly centralised, or operationally difficult to maintain.

This issue is directly relevant to the present research. Where many services rely on shared platforms such as data stores, gateways, or infrastructure layers, the failure of one component may create disproportionate systemic disruption. Such patterns suggest that architectural quality should be evaluated not only by modularity, but also by resilience under failure conditions.

Research into fault tolerance strategies addresses this challenge from an operational perspective. (Mailewa et al., 2025) identify techniques such as retries, circuit breakers, replication, and graceful degradation as common methods for improving service continuity. These mechanisms are valuable because they reduce outage severity and improve recovery capability.

Nevertheless, these methods primarily focus on response after faults occur. They do not always provide a framework for estimating which services represent the highest pre-failure systemic risk. This distinction is important because prevention and prioritisation decisions often need to be made before incidents arise.

3.3 Depth and Breadth of Coverage

The literature demonstrates that cascading failure risk in microservices cannot be understood through one discipline alone. Instead, the topic spans several interconnected research areas.

First, software architecture literature explains why microservices are adopted and how distributed decomposition changes system design. Second, reliability engineering literature provides metrics such as failure rate, Mean Time to Recovery (MTTR), availability, and redundancy planning. Third, resilience engineering examines how systems tolerate disruption through circuit breakers, retries, failover controls, and graceful degradation. Fourth, chaos engineering investigates how systems

behave when faults are deliberately introduced under controlled conditions. Fifth, graph theory offers mathematical tools for analysing dependency networks and propagation pathways.

This breadth is important because each field contributes only part of the wider problem. Microservice studies explain the environment. Reliability studies explain component behaviour. Graph studies explain structure. Resilience studies explain mitigation. However, no single stream fully explains how to quantify system-wide cascading impact in an interpretable and efficient manner.

The literature also reflects increasing concern regarding dependency complexity. As systems scale, it becomes harder to identify hidden single points of failure, shared infrastructure bottlenecks, circular dependencies, or recovery constraints. This suggests that future resilience research must increasingly combine architectural understanding with quantitative analysis.

3.4 Comparative Analysis

A comparison of current approaches highlights both strengths and limitations.

Architectural studies provide valuable guidance on decomposition, team autonomy, and scalability. Their strength lies in helping organisations design modern systems. However, they are less effective at quantifying outage impact or ranking critical dependencies once systems become operational.

Fault tolerance literature provides practical controls such as retries, replication, circuit breakers, and fallback behaviour. These methods are highly useful in production environments, but they often operate at local service scope rather than architecture-wide scope. A service may be technically resilient while still remaining a major systemic dependency.

Chaos engineering offers realistic testing because it evaluates actual system behaviour under disruption. This can provide stronger empirical evidence than purely theoretical approaches. However, chaos testing may require mature infrastructure, extensive monitoring, and organisational tolerance for controlled failure events. Repeated experimentation across many scenarios may also be resource intensive.

Graph-based approaches are valuable because they reveal how failure can propagate structurally through interconnected systems. Central nodes, upstream dependencies, and bottlenecks can be identified more clearly than through isolated component metrics. O'Halloran et al. (2021) demonstrate how graph techniques can support failure propagation analysis in engineered systems.

Traditional reliability metrics such as MTTR and failure rate are also important, but when used alone, they may ignore dependency context. A component with modest failure frequency may still create severe disruption if many other services rely upon it. This suggests that metrics become more meaningful when interpreted within dependency networks.

Overall, current research often treats structure, reliability, and resilience as separate concerns rather than integrated dimensions of one risk problem.

3.5 Identification of Gaps

The principal gap identified through the literature is the limited availability of lightweight analytical frameworks that combine service dependency topology with operational reliability metrics to estimate cascading failure impact.

Existing research tends to focus on one of the following areas:

- how microservices should be designed
- how failures should be tolerated
- how disruption can be experimentally tested
- how networks can be structurally analysed
- how component reliability can be measured

Fewer studies combine these perspectives into a practical framework capable of answering questions such as:

- Which service failure would create the greatest system-wide disruption?
- How does increased recovery time affect total vulnerability?
- Does reducing dependency propagation significantly improve resilience?
- Which components should receive resilience investment first?

This gap is important because organisations increasingly need proactive decision-support tools rather than relying solely on post-incident learning or expensive experimentation.

3.6 Appropriate Sources and Quality

The sources reviewed in this chapter were selected to ensure academic credibility and relevance. They include peer-reviewed journal articles, respected conference papers, recognised professional literature, and practical industry sources relevant to distributed systems.

For example, Velepucha and Flores (2023) provide recent survey evidence on microservice architectures. Taibi and Lenarduzzi (2018) contribute insight into recurring microservice design weaknesses. (Mailewa et al., 2025) offer current discussion of resilience testing and chaos

engineering. O'Halloran et al. (2021) provide graph-based propagation analysis, while Avižienis et al. (2004) offer foundational dependability theory.

Together, these sources provide both contemporary relevance and theoretical depth, supporting a balanced and authoritative review.

3.7 Relation to This Research and Hypothesis

The reviewed literature strongly supports the need for the present study. Existing research confirms that microservice architectures increase dependency complexity, that failures can propagate across distributed systems, and that resilience mechanisms are necessary for maintaining service continuity. However, the literature also indicates that many current approaches focus separately on architecture design, fault mitigation, or resilience testing rather than providing an integrated method for measuring overall system vulnerability.

This dissertation responds to that gap through the following hypothesis:

A graph-based analytical model that combines dependency topology with operational reliability metrics can provide a practical and interpretable method for quantifying cascading failure impact and system risk in cloud microservice architectures.

To examine this hypothesis, the study integrates directed dependency graphs with key operational variables, including failure rate, propagation probability, and Mean Time to Recovery (MTTR). This enables service failures to be analysed not only as isolated technical events, but as potential sources of wider architectural disruption.

The study also considers whether recurring dependency concentration may indicate structural weaknesses similar to recognised microservice bad smells. Where a small number of shared services dominate risk exposure, this may suggest opportunities for redesign, redundancy improvement, or dependency reduction.

Initial scenario-based findings provide support for the hypothesis. The model distinguished between lower-risk and higher-risk services, identified recurring critical components across multiple operating scenarios, and demonstrated that increases in MTTR or failure frequency produced higher overall System Vulnerability Index (SVI) values.

These findings suggest that graph-based analytical modelling can extend existing literature by linking system structure, operational reliability, and measurable risk outputs within one efficient framework. With broader validation using real-world datasets, the approach could provide practical value for resilience planning, migration assessment, and proactive reliability management.

4 Methodology

4.1 Introduction

This chapter explains the methodological approach used to design, implement, test, and evaluate the proposed framework for quantifying cascading failure impact in cloud microservice architectures. It describes the research design, system modelling process, data inputs, implementation choices, scenario experimentation, testing strategy, validation methods, and practical considerations.

The purpose of the methodology was to develop a repeatable and interpretable framework capable of assessing how service dependencies and reliability conditions influence system-wide vulnerability.

To achieve this, a graph-based software artefact was developed in Java using configurable CSV datasets and scenario testing. The final implementation prioritised transparency, reproducibility, and efficient evaluation rather than unnecessary computational complexity.

4.2 Research Design and Choice of Methods

This study adopted a **quantitative experimental modelling approach**. Rather than collecting human-subject data or deploying live fault injection in production systems, the research focused on constructing an analytical model of service dependencies and testing its behaviour under controlled scenarios.

This method was selected for three main reasons.

First, the research question concerns system-level risk estimation rather than user perception or organisational behaviour. Quantitative modelling was therefore more suitable than interviews or surveys.

Second, direct experimentation on real production cloud systems would involve cost, access restrictions, and operational risk. Controlled synthetic modelling allowed failure behaviour to be studied safely and repeatably.

Third, the project required interpretable outputs such as service rankings, expected impact values, and vulnerability indicators. Analytical modelling was chosen because it generates deterministic results for a given input set, enabling clear comparison between scenarios.

The overall methodology followed an iterative engineering cycle:

1. Review literature on microservice reliability and failure propagation
2. Design dependency graph architecture

3. Construct input datasets
4. Implement an analytical engine in Java
5. Run controlled scenarios
6. Generate outputs and visualisations
7. Evaluate findings against research objectives

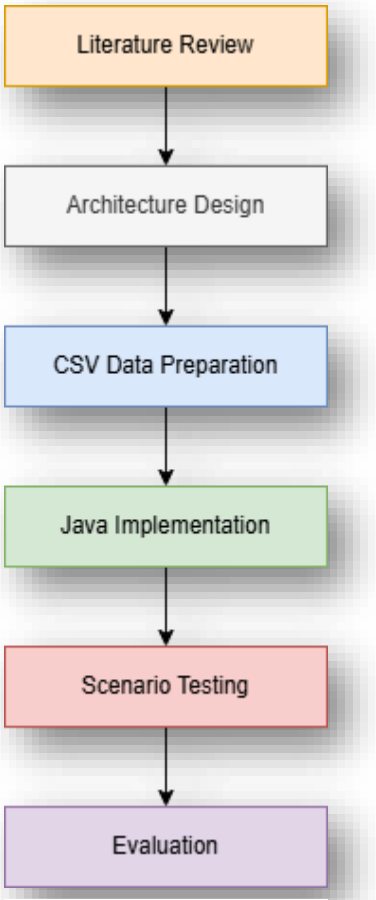

Figure 1 Methodology Workflow

4.3 System Architecture and Data Model

The proposed framework models a cloud microservice environment as a directed dependency graph. In this representation:

Nodes represent application services or infrastructure components

Edges represent dependency relationships between components

Edge weights represent propagation probability between connected services

If Service A depends on Service B, failure of Service B may adversely affect Service A. This relationship is therefore represented as a directed edge within the graph structure.

Graph modelling was selected because it provides a clear and interpretable way to represent complex service dependencies while enabling quantitative analysis of cascading disruption paths. It also supports scenario testing by allowing parameter values to be adjusted without redesigning the underlying architecture.

A realistic distributed architecture was required to make the model meaningful. Therefore, the system design was derived from a publicly available Twitter-style distributed system reference architecture and mirrored into equivalent Google Cloud Platform (GCP) services.

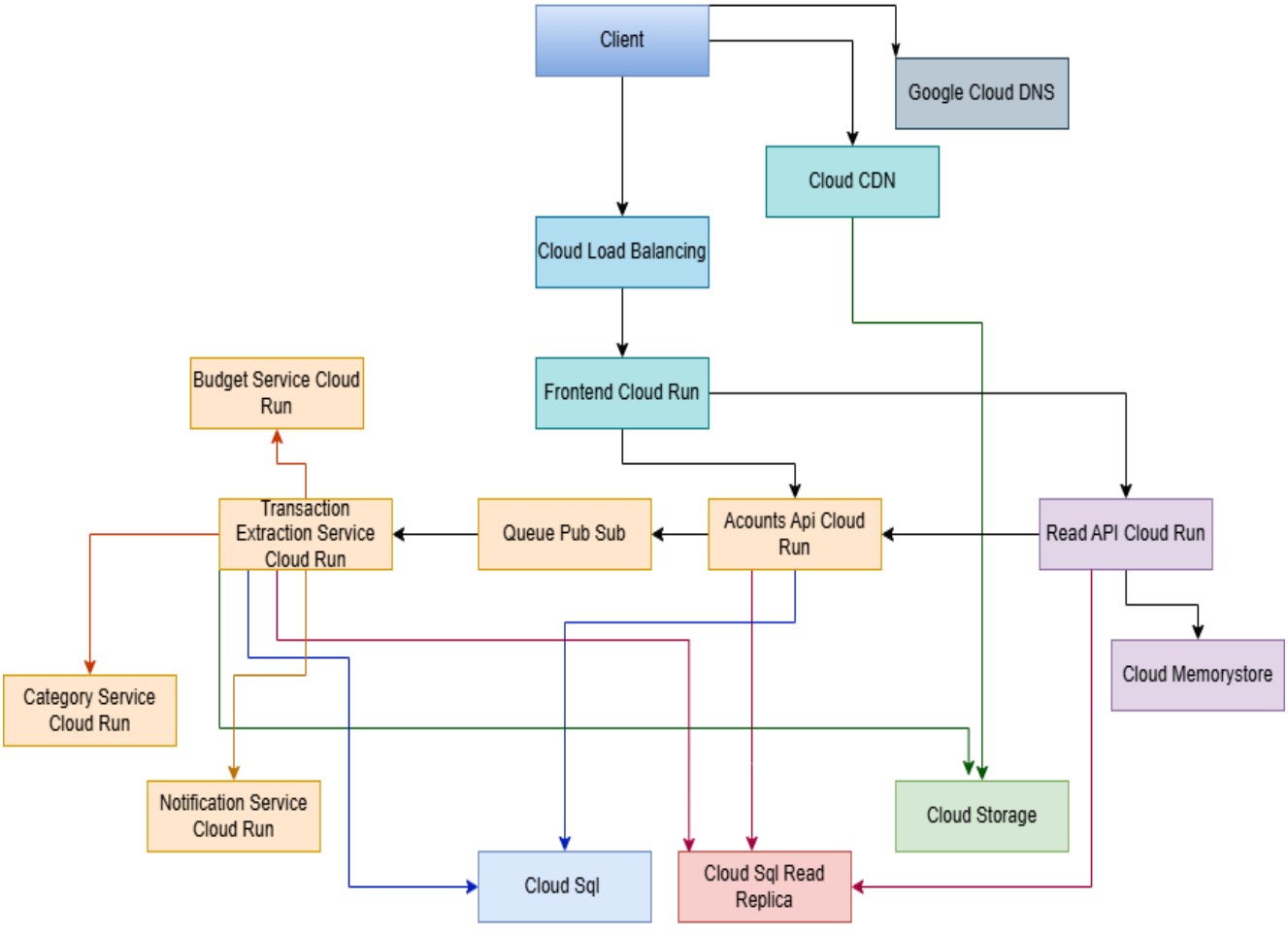

Figure 2 GCP Mirrored Dependency Architecture (author's own work)

4.4 Twitter to GCP Mapping

A known reference architecture was selected because it provides a realistic multi-service dependency structure involving frontend delivery, application services, data storage, caching, and traffic management layers. Such characteristics closely resemble many modern cloud-native production systems.

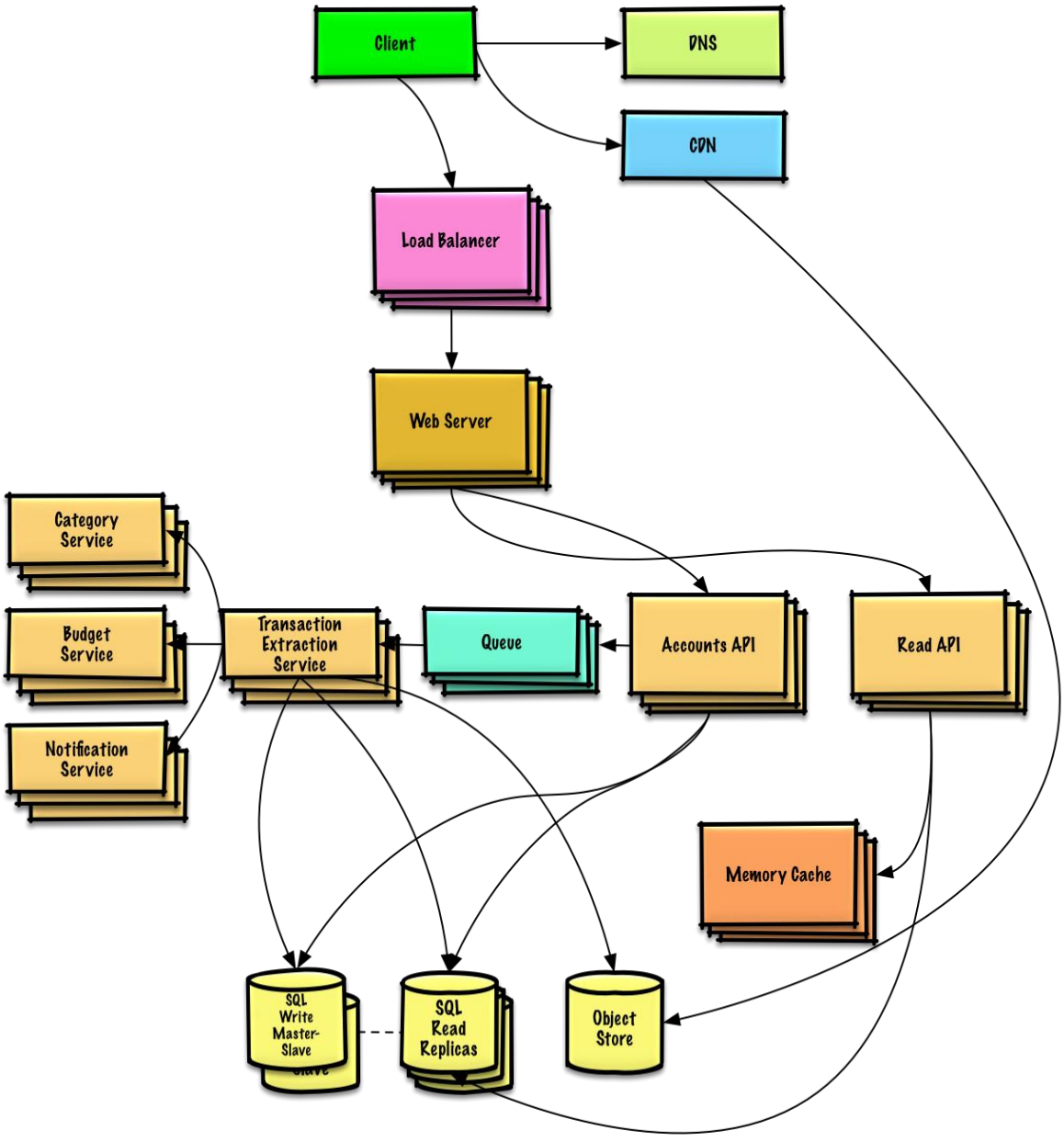

Figure 3 Twitter Reference Architecture (Source: Donne Martin (n.d.), System Design Primer)

Equivalent managed GCP services were then used to modernise the design and align it with publicly documented cloud service incidents and operational environments. This improved the practical relevance of the model while retaining the dependency logic of the original design.

Table 1 Twitter Architecture Components Mapped to GCP Services

Reference Component	Mirrored GCP Service
DNS	Google Cloud DNS
CDN	Cloud CDN
Load Balancer	Cloud Load Balancing
Web Tier	Frontend Cloud Run
API Services	Accounts API / Read API Cloud Run
Queue	Pub/Sub
Cache	Memorystore
SQL Database	Cloud SQL
Read Replica	Cloud SQL Read Replica
Object Storage	Cloud Storage

GCP services were also selected because publicly documented Google Cloud service incidents provided real-world examples of platform disruption, allowing the study to relate synthetic scenario modelling to realistic cloud reliability events.

Equivalent managed GCP services were then used to modernise the design and align it with publicly documented cloud service incidents and operational environments. GCP services were also selected because documented Google Cloud outage reports provided practical reference points for understanding how infrastructure failures can affect dependent services. This improved the realism of the model while retaining the dependency logic of the original architecture.

4.5 Data Inputs and CSV Configuration

The model was designed for reproducibility using CSV input files rather than hard-coded values. Separate files defined:

- **Services file** – service names, failure rates, MTTR values
- **Dependencies file** – source service, target service, propagation probability
- **Scenario files** – modified parameters for scenario testing

This approach offered several benefits:

- transparent parameter management
- rapid scenario switching
- easier debugging
- repeatable experimentation
- separation of data from code

Example scenario configuration files and dataset inputs are provided in **Appendix B**.

Table 2 Example Service Parameters

Service	Failure Rate	MTTR	Role
Cloud DNS	0.01	5	Routing
Frontend Cloud Run	0.03	10	User entry point
Accounts API	0.04	15	Business logic
Cloud SQL	0.07	30	Primary database
Memorystore	0.03	10	Cache

Table 3 Example Dependency Parameters

Source	Target	Propagation Probability
Frontend	Load Balancer	0.95
Accounts API	Frontend	0.90
Cloud SQL	Accounts API	0.90
Cache	Read API	0.85

4.6 Analytical Impact Model

The final implementation used a custom Java **ImpactCalculator** engine to estimate cascading failure impact across the dependency graph. The objective of the model was to quantify how failure of one service may influence downstream components and contribute to wider system vulnerability.

For each source service, the engine performs the following steps:

1. Assign an initial failure probability of **1.0** to the source service
2. Traverse all reachable downstream dependencies
3. Multiply propagation probabilities along dependency paths
4. Estimate downtime impact using the MTTR of affected services
5. Calculate service-level and system-level risk metrics

This analytical approach was selected because it provides deterministic, repeatable outputs for a given dataset, allowing consistent comparison across multiple scenarios.

Core Measures

The model calculates the following outputs:

- **Dependent Service Loss (DSL):** cumulative expected downtime imposed on affected downstream services
- **Expected System Impact (ESI):** source service failure rate weighted by downstream loss
- **Affected Service Ratio (ASR):** proportion of other services indirectly affected
- **System Vulnerability Index (SVI):** total aggregated vulnerability across all services

- **Maximum Risk Service (MRS):** service with the highest expected impact score

Strongest-Path Assumption

Where multiple dependency paths could reach the same downstream service, the implementation retained the highest-probability discovered path.

This assumption was adopted to:

- avoid double counting the same affected service
- preserve interpretability
- reduce modelling complexity
- support efficient scenario comparison

Although simplified, this provided a practical estimate of cascading influence suitable for decision-support analysis.

Strongest-Path Logic

A → B → D = 0.72
A → C → D = 0.54
Retain A → B → D

- **Analytical Impact Formulation**

The initial conceptual model estimated the impact of a failed service as the cumulative disruption imposed on reachable downstream services:

$$Impact(s) = \sum_{v \in Reachable(s)} FP(s, v) \times MTTR(v)$$

Where:

- $FP(s, v)$represents the propagation probability from the service s to service v
- $MTTR(v)$represents the recovery time of the affected service v

To better reflect operational conditions, the final model was extended to include service failure likelihood and multi-hop dependency propagation:

$$Impact(s) = \lambda_s \times \sum_{v \in Reachable(s)} \left(FP_{path}(s, v) \times MTTR(v)\right)$$

Where:

- λ_s= failure rate of service s
- $FP_{path}(s, v)$= cumulative propagation probability from $s$to v

For a dependency chain:

$$s \rightarrow a \rightarrow b \rightarrow v$$

$$FP_{path}(s, v) = FP(s, a) \times FP(a, b) \times FP(b, v)$$

This extension improves realism by combining failure frequency, propagation likelihood, and recovery duration within a single impact measure.

Selected extracts from the Java implementation are provided in **Appendix A**.

4.7 Earlier Simulation Prototype

Before the final analytical engine, an earlier stochastic prototype (FailureEngine) was developed using Monte Carlo simulation.

This engine:

- randomly triggered direct failures using failure rates
- propagated failures probabilistically through dependencies
- repeated iterations many times
- recorded downtime and cascaded failures

The prototype was useful for understanding propagation dynamics and validating graph traversal logic. However, it was not adopted as the final evaluation method because repeated runs produced variable outputs and required higher computational effort.

The analytical model was therefore selected as the final framework due to greater repeatability and clearer interpretation.

4.8 Scenario Design

Five scenarios were designed to isolate key reliability variables.

Table 4 Scenario Definitions

Scenario	Purpose
Baseline	Standard operating assumptions
Low Probability	Reduced propagation strength
High Probability	Increased propagation strength
High MTTR	Slower service recovery
High Failure Rate	Increased likelihood of direct failure

These scenarios were selected because they directly test the variables central to the research model:

- propagation probability
- recovery duration
- failure frequency

4.9 Tools and Implementation Technologies

Table 5 Tools Used

Tool	Purpose
Java	Core implementation
IntelliJ IDEA	Development environment
CSV Files	Configurable datasets
R	Statistical plotting and charts
GitHub Source Reference	Architecture inspiration
Spreadsheet Tools	Input review and output inspection

Java was selected as the primary implementation language because it provides strong object-oriented modelling, suitable support for graph-based data structures, and efficient file processing for structured CSV datasets. These characteristics made it appropriate for representing services, dependencies, and analytical calculations within a modular software design. R was used for post-

processing and visualisation because it enables clear production of comparative charts, including System Vulnerability Index (SVI) comparisons, service impact rankings, and scenario trend graphs.

4.10 Testing Strategy

Testing was designed to verify the correctness, consistency, and practical usability of the analytical impact model. As the project artefact was a computational research tool rather than an end-user application, testing focused on model logic, data handling, scenario sensitivity, and stable execution rather than graphical interface testing.

Initial testing was performed on the CSV input process to ensure that service records, dependency relationships, and scenario files were loaded correctly into the graph structure. This included checking that services were created successfully, dependencies were linked to the correct nodes, and incomplete or invalid records could be identified during data preparation.

Functional testing was then carried out on the calculation engine. Controlled parameter changes were introduced to confirm that outputs responded logically to altered reliability conditions. For example, increasing Mean Time to Recovery (MTTR) was expected to increase total vulnerability because longer outages generate greater disruption. Increasing service failure rates was expected to raise expected impact values, while reducing propagation probabilities was expected to lower cascading reach and dependent service loss. Scenario outputs followed these expected patterns, indicating that the model calculations were operating correctly.

Repeatability testing was also important. Because the final implementation uses deterministic analytical calculations rather than random simulation, repeated execution using the same input datasets produced identical rankings, impact values, and System Vulnerability Index (SVI) results. This confirmed that the framework was suitable for consistent scenario comparison.

Execution testing was conducted by running all five scenarios—Baseline, Low Probability, High Probability, High MTTR, and High Failure Rate—to ensure that the system completed successfully without runtime errors. Output CSV files and chart data were generated correctly for each scenario.

Overall, the testing strategy demonstrated that the software implementation operated consistently, processed inputs correctly, and responded logically to changes in reliability parameters.

4.11 Validation and Reliability

Validation focused on assessing whether the model outputs were credible, logically consistent, and aligned with theoretical expectations from the literature on distributed systems and cascading failures. As the project used synthetic datasets and an analytical framework rather than live

production telemetry, validation should be interpreted as indicative support for model usefulness rather than definitive empirical proof.

The first validation approach was **structural validation**. Services positioned upstream in the dependency graph, such as routing and traffic-distribution components, would reasonably be expected to affect a larger proportion of downstream services when disrupted. The results were consistent with this expectation, as infrastructure-layer services generally produced higher affected-service ratios than more isolated downstream business services.

The second approach was **risk-ranking validation**. Shared backend platforms, particularly data-layer services with multiple dependants, would be expected to generate comparatively high system impact because disruption may influence several connected components simultaneously. This pattern was reflected in the scenario outputs, where shared platform services repeatedly appeared among the higher-risk components.

The third approach was **scenario sensitivity validation**. If the model was responding logically to changing reliability conditions, scenarios with higher MTTR or higher failure rates should increase overall vulnerability relative to the baseline case. Conversely, reduced propagation probabilities should lower total cascading impact. The generated outputs followed these expected directional trends, providing evidence that the framework was sensitive to meaningful parameter changes.

The fourth approach was **consistency validation**. Several services remained highly ranked across multiple scenarios, suggesting that the model was identifying persistent structural risk rather than producing unstable or arbitrary results.

Reliability was further supported by the deterministic nature of the final analytical engine. Unlike the earlier Monte Carlo prototype, the implemented model does not rely on random sampling and therefore produces identical outputs whenever the same inputs are used. This characteristic improves repeatability and supports controlled scenario comparison.

However, validation remains subject to limitations. The model uses synthetic parameter values and simplified assumptions, including static propagation probabilities and the strongest-path approximation. Consequently, the framework should be regarded as a decision-support tool for comparative risk assessment rather than an exact representation of production cloud behaviour. Further validation using operational telemetry or historical incident datasets would strengthen confidence in future versions of the model.

4.12 Ethical, Legal, Social, and Professional Considerations

No human participants, surveys, or personal data were used in this project. As a result, formal human-subject ethical risk was minimal. The datasets used were synthetic or based on publicly

available architectural abstractions, and no sensitive operational logs, customer records, or confidential cloud-provider data were processed during the study.

If future versions of the research incorporate real incident logs or production telemetry, stronger governance controls would be required. These would include anonymisation of identifiers, secure data storage, collection of only the minimum necessary fields, and compliance with relevant legal and organisational policies such as GDPR requirements.

From a professional perspective, the study aims to support reliability engineering practice through proactive risk assessment rather than purely reactive outage response. By helping organisations identify structurally important dependencies in advance, similar methods could contribute to more resilient digital service delivery.

4.13 Practicality, Constraints and Limitations

The methodology was intentionally designed to remain feasible within the available time, technical scope, and project resources. A key practical advantage of the approach is that it enables low-cost experimentation without requiring access to enterprise production systems or disruptive live testing environments. The use of configurable datasets also supported repeatable scenario analysis, while the analytical outputs remained interpretable for decision-making purposes.

However, several constraints should be recognised. Reliability parameters such as failure rates, recovery times, and propagation probabilities were synthetic estimates rather than values derived from proprietary operational datasets. The model also assumes static probabilities during each scenario and applies a strongest-path approximation when multiple dependency routes exist. In addition, no live telemetry or continuous monitoring feeds were integrated, meaning that dynamic production behaviour could not be captured directly.

These limitations were considered acceptable because the objective of the project was to develop a practical decision-support model rather than a full digital twin of a real cloud platform. The framework is therefore better interpreted as a comparative risk-analysis tool than an exact predictor of operational incidents.

4.14 Chapter Summary

This chapter explained the methodology used to develop and evaluate the proposed cascading failure impact framework. A directed dependency graph model was implemented in Java using configurable CSV datasets and tested across five reliability scenarios. The final analytical engine was selected over an earlier Monte Carlo prototype because it provided repeatable, interpretable, and efficient outputs.

Testing and validation indicated that the framework behaved logically under changing reliability conditions and produced consistent comparative results. Ethical risks were minimal due to the absence of personal or confidential operational data, while practical limitations were acknowledged through the use of synthetic parameters and simplified modelling assumptions.

The next chapter presents the results of scenario experimentation and evaluates the effectiveness of the model in identifying critical services and measuring system vulnerability.

5 Quality and Results

5.1 Introduction

This chapter presents the results of the graph-based analytical framework and evaluates the quality of the implemented solution. The purpose of the chapter is not only to report numerical outputs, but also to critically interpret what those outputs indicate in relation to cascading failure risk, service criticality, and system-wide vulnerability in cloud microservice architectures.

The framework was tested using five scenarios: **Baseline**, **Low Propagation Probability**, **High Propagation Probability**, **High MTTR**, and **High Failure Rate**. Outputs were generated using the Java implementation and visualised through R. These results are analysed against the research objectives established earlier in the dissertation.

5.2 Metrics and Presentation

To evaluate the model, five core metrics were used. These metrics were selected because they support analysis at both service level and whole-system level, enabling the study to identify critical components while also comparing wider operating conditions.

Selected R scripts used for comparative visualisation are included in **Appendix D**.

Expected System Impact measures the expected disruption caused by the failure of an individual service. It combines failure likelihood with estimated downstream impact.

Dependent Service Loss represents the cumulative expected downtime imposed on affected downstream services after propagation.

Affected Service Ratio measures the proportion of the architecture indirectly affected by the failure of a service and therefore reflects the structural blast radius.

Maximum Risk Service identifies the service with the highest Expected System Impact within a scenario.

System Vulnerability Index (SVI) is the sum of all service impact values across the architecture and provides a comparative indicator of total vulnerability.

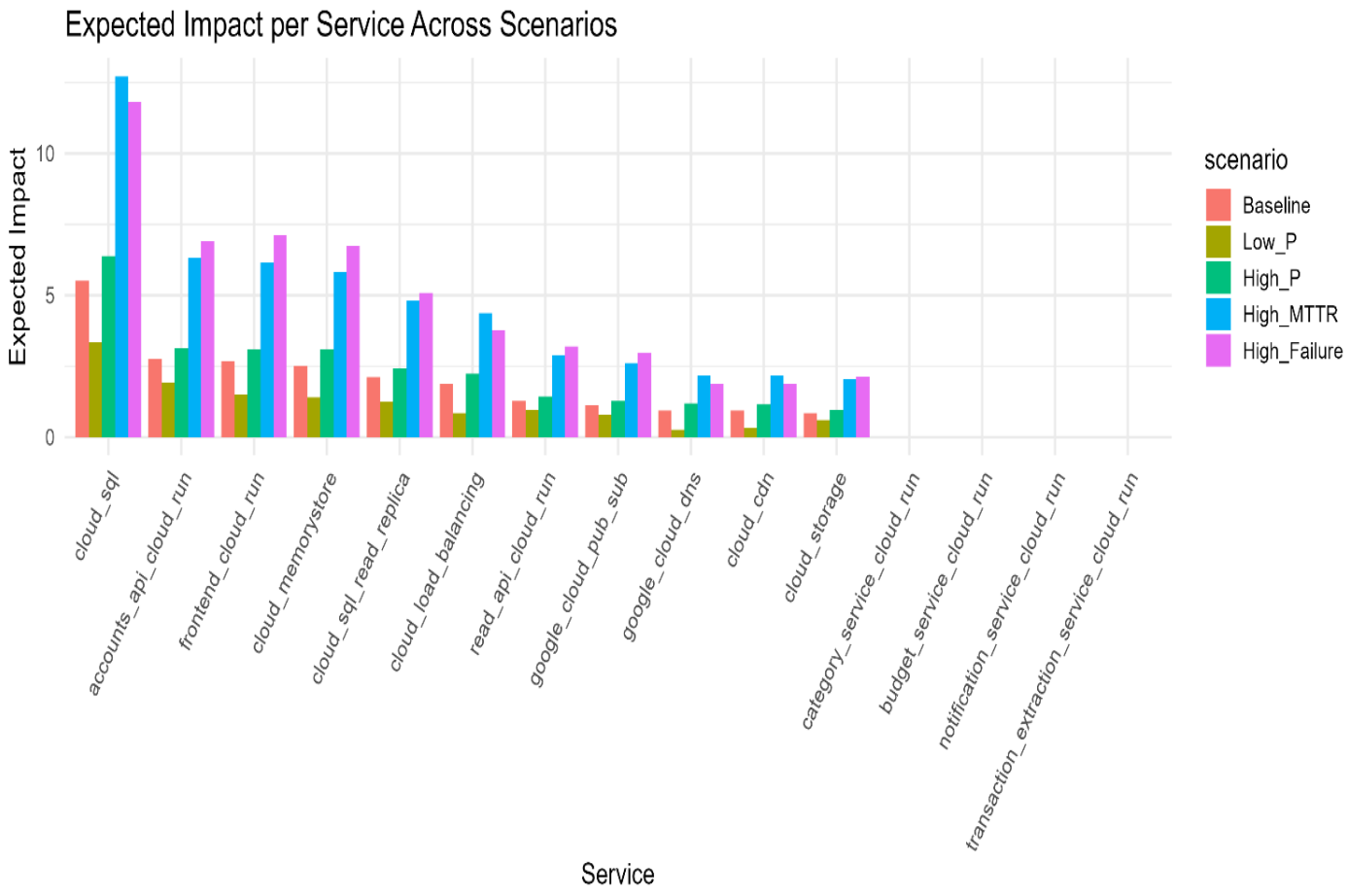

Figure 4 Expected Impact by Service

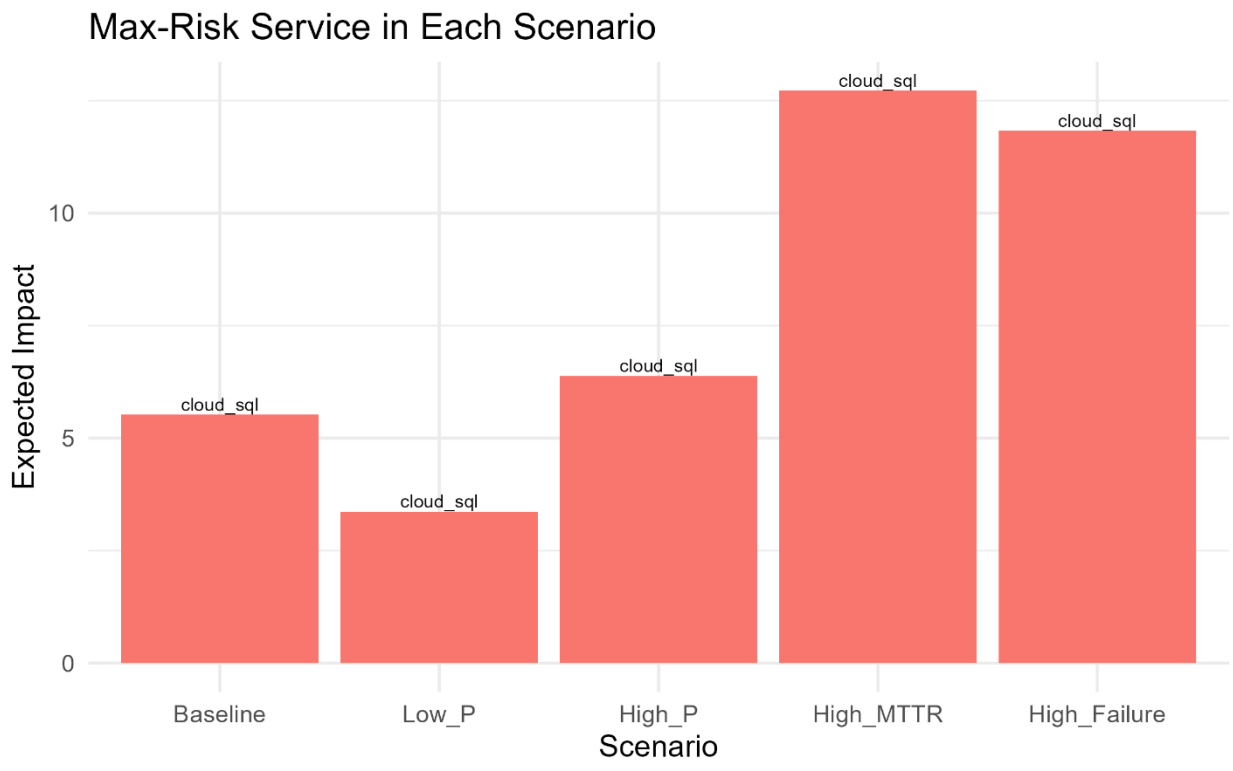

Figure 5 Maximum Risk Service

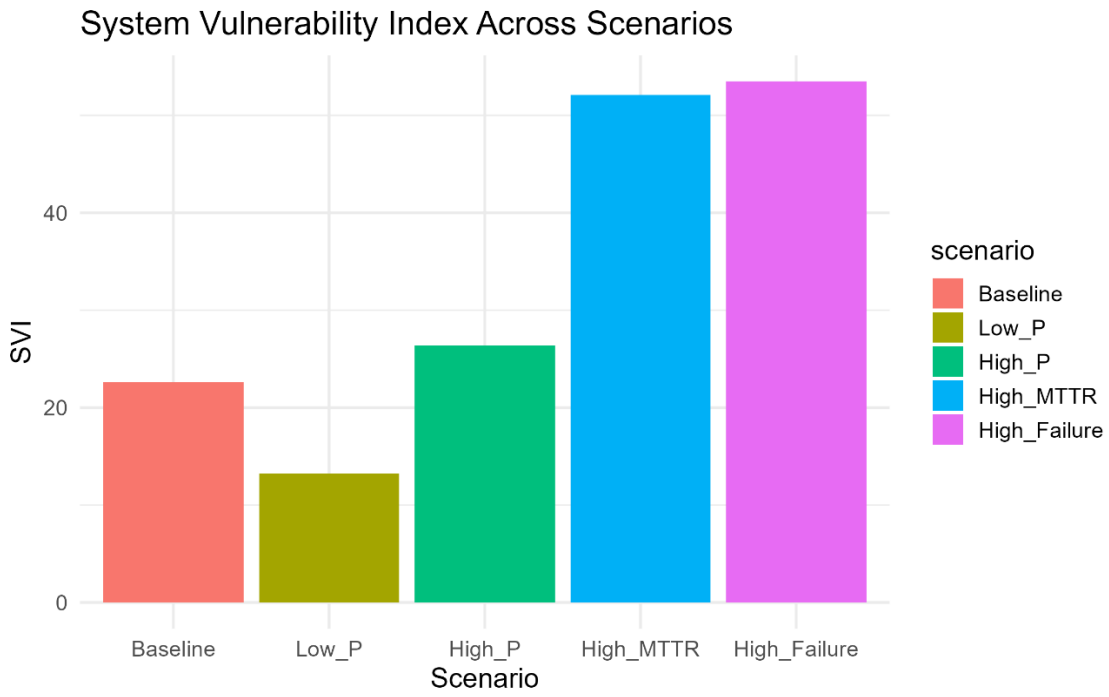

Figure 6 SVI Comparison

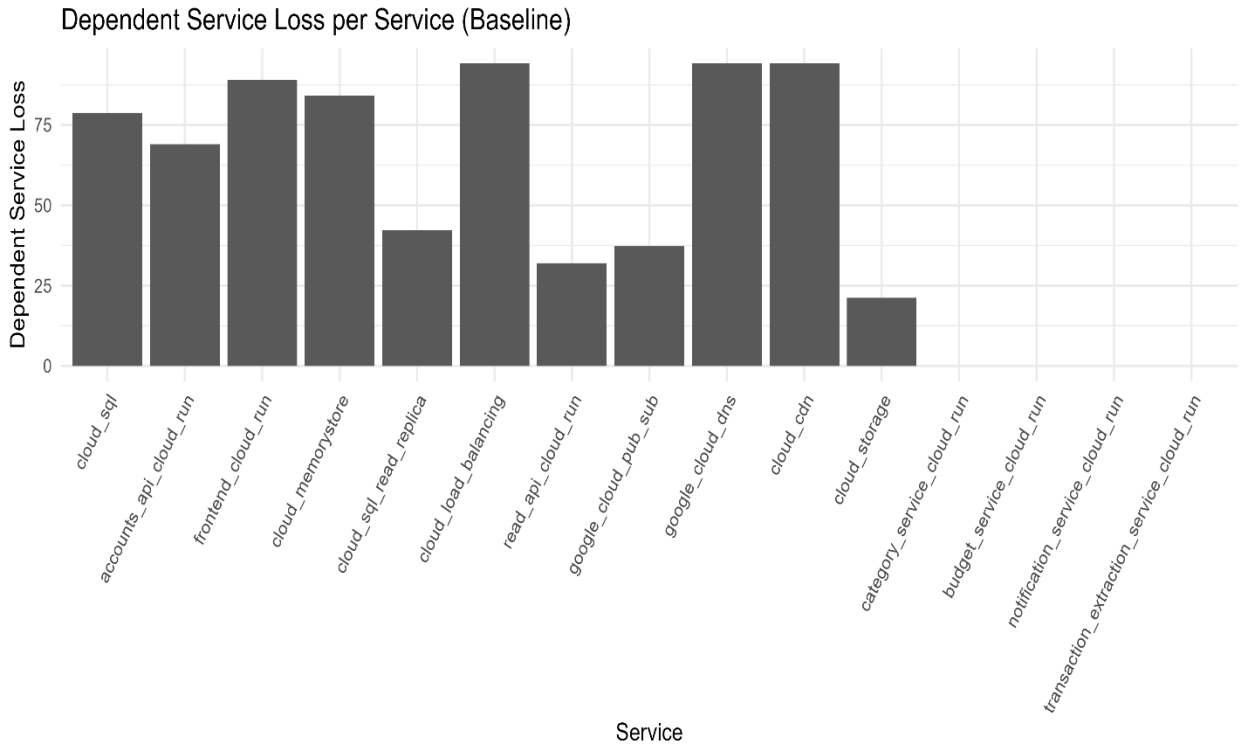

Figure 7 Dependent Service Loss

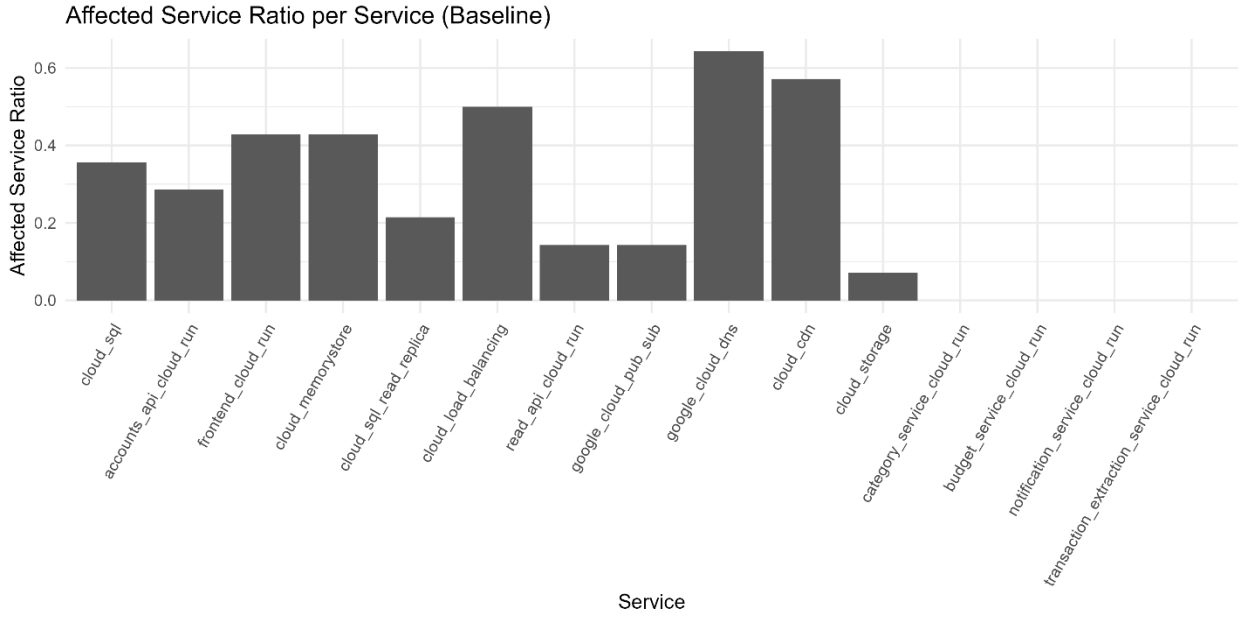

Figure 8 Affected Service Ratio

5.3 Critical Analysis

The scenario outputs showed clear and logically consistent variation across the tested operating conditions.

The **High Failure Rate** scenario produced the highest overall vulnerability because increased direct failure likelihood amplified every downstream impact pathway. This indicates that frequent component instability may create greater systemic disruption than isolated severe outages.

The **High MTTR** scenario also significantly increased vulnerability. Longer recovery duration increases disruption persistence, meaning even moderate propagation becomes costly when affected services remain unavailable for extended periods. This is consistent with dependability literature, where repair and recovery performance strongly influence resilience (Avižienis et al., 2004).

The **Low Propagation Probability** scenario reduced system-wide impact relative to the baseline. This suggests that architectural isolation, fault containment, and graceful degradation can materially reduce cascading risk.

The **High Propagation Probability** scenario increased SVI above baseline levels, confirming that tightly coupled service ecosystems are more vulnerable to widespread disruption.

Overall, these results support the central hypothesis that cascading risk is shaped not by one variable alone, but by the interaction of failure likelihood, recovery duration, and dependency structure.

5.4 Evidence of Practical Work

This project involved substantial practical implementation rather than theoretical discussion alone.

In addition, an earlier Monte Carlo simulation prototype was developed before the final analytical engine was selected. This prototype simulated random failures and probabilistic propagation, helping validate the conceptual model before transitioning to the final deterministic framework.

The practical artefact therefore, evolved through iterative development rather than being produced in a single stage.

The completed artefact included a custom Java graph engine for service modelling, CSV-based parameter datasets, dependency graph construction logic, an analytical impact calculation engine, scenario execution across five operating conditions, automated CSV export of outputs, and R-generated comparative charts.

impact_high_failure.csv | impact_high_p.csv | impact_high_mttr.csv | impact_low_p.csv | si

```
service,dependent_service_loss,self_downtime,expected_system_impact,affected_service_ratio,af
google_cloud_dns,94.22836249999999,5.0,1.88456725,0.6428571428571429,9
cloud_cdn,94.18775000000001,5.0,1.8837550000000003,0.5714285714285714,8
cloud_load_balancing,94.145,5.0,3.7658,0.5,7
frontend_cloud_run,89.10000000000001,10.0,7.128000000000001,0.4285714285714285,6
accounts_api_cloud_run,69.0,15.0,6.9,0.2857142857142857,4
read_api_cloud_run,32.0,15.0,3.2,0.14285714285714285,2
category_service_cloud_run,0.0,20.0,0.0,0.0,0
budget_service_cloud_run,0.0,20.0,0.0,0.0,0
notification_service_cloud_run,0.0,20.0,0.0,0.0,0
transaction_extraction_service_cloud_run,0.0,25.0,0.0,0.0,0
cloud_memorystore,84.15,10.0,6.732,0.4285714285714285,6
queue_pub_sub,37.25,10.0,2.98,0.14285714285714285,2
cloud_sql,78.85,30.0,11.827499999999999,0.35714285714285715,5
cloud_sql_read_replica,42.300000000000004,25.0,5.0760000000000005,0.21428571428571427,3
cloud_storage,21.25,20.0,2.125,0.07142857142857142,1
```

Figure 9 Scenario Output CSV Generated by Java Implementation

Figure 9 demonstrates that the project produced structured outputs suitable for further analysis rather than only conceptual discussion. The exported files enabled consistent scenario comparison and direct integration with the visualisation stage.

Example scenario outputs generated by the analytical engine are shown in **Appendix C**.

5.5 Technical Challenges and Solutions

Several technical challenges emerged during implementation.

A major challenge involved **multi-path overcounting**. Where multiple dependency routes could reach the same service, naïve path summation risked inflating impact values. This was addressed through a strongest-path approximation, retaining the highest-probability discovered route to each service.

A second challenge involved **simulation variability**. The earlier Monte Carlo prototype produced changing outputs across repeated runs, making scenario comparison more difficult. This was resolved by adopting the final analytical model, which generates repeatable outputs for identical inputs.

A third challenge concerned **realistic architecture modelling**. Purely synthetic graphs risk limited practical relevance. To improve realism, an established distributed system reference architecture was mirrored into cloud-managed services aligned with publicly documented platform environments.

These solutions improved interpretability, repeatability, and practical value.

5.6 Novelty and Innovation

The originality of this project lies in integrating several research strands into one lightweight decision-support framework.

Existing studies often focus separately on microservice architecture design, resilience mechanisms, chaos engineering, graph theory, or isolated reliability metrics. This project combines directed dependency topology, failure rate, propagation probability, MTTR, service-level risk ranking, and system-wide vulnerability scoring within one analytical model.

The framework therefore offers a useful middle ground between oversimplified structural analysis and operationally expensive resilience experimentation. It can be applied without requiring live fault injection, confidential enterprise telemetry, or large-scale simulation resources.

5.7 Interpretation of Results

The results provide several important insights.

First, shared backend platforms repeatedly emerged as high-risk components. This is expected because they support multiple downstream services and may involve longer recovery times. In practical environments, this implies that resilience investment in data-layer redundancy, replication, and failover capability may provide strong returns.

Second, upstream infrastructure services such as routing or traffic-distribution layers showed high affected-service ratios. Their failure may not always generate the largest downtime score, but they can influence a broad portion of the architecture.

Third, changes in MTTR had a strong effect on vulnerability. This suggests that faster incident response, automation, and recovery processes may deliver resilience gains comparable to reducing raw failure frequency.

Fourth, recurring critical services remained highly ranked across multiple scenarios. This indicates that the model was identifying persistent structural importance rather than unstable or random outputs.

These findings are also consistent with publicly reported cloud incidents in which disruption to shared infrastructure affected multiple dependent services simultaneously.

5.8 Tools and Techniques

The selected tools proved appropriate for the project objectives.

Java was effective for implementing graph structures, object-oriented service models, and deterministic calculations. It also handled scenario batch execution efficiently.

CSV files provided a transparent and editable method for configuring services, dependencies, and scenario variations.

R was highly suitable for chart generation and comparative visualisation of outputs such as SVI and impact rankings.

Although effective, the toolchain remained offline and static. It did not integrate with live observability platforms, cloud APIs, or streaming telemetry. However, for a controlled research project focused on modelling rather than production deployment, the selected tools were proportionate and practical.

5.9 Links to Objectives and Literature

The results directly support the original project objectives.

The objective of representing a microservice system as a dependency graph was achieved through the implemented node-edge architectural model. The objective of incorporating reliability parameters was met through the use of failure rate, propagation probability, and MTTR inputs.

The objective of estimating cascading failure impact was achieved through service-level impact calculations, dependent service loss outputs, and affected-service ratios.

The objective of comparing vulnerability across scenarios was achieved through five controlled scenario runs and comparative SVI analysis.

The objective of identifying critical services was achieved through Maximum Risk Service rankings and recurring high-impact components across scenarios.

The findings also align with prior literature. Graph-based studies emphasise the importance of structural centrality and propagation pathways (O'Halloran et al., 2021), while resilience research highlights recovery performance and fault containment (Mailewa et al., 2025)). This study contributes by integrating those perspectives within one comparative framework.

5.10 Feasibility and Realism

The framework proved realistic within project constraints. It required no access to confidential enterprise systems, no live production fault injection, no expensive infrastructure, and only modest computing resources.

At the same time, it produced meaningful comparative outputs using dependency structures similar to real cloud deployments. The use of cloud-managed service equivalents also improved realism by aligning the model with a platform that has publicly documented incident history.

This suggests the framework could be useful during architecture reviews, migration planning, resilience prioritisation, educational risk analysis, and pre-deployment design assessment.

However, realism remains bounded by synthetic parameters and simplified assumptions.

5.11 Additional Findings

A key emergent finding was that service importance cannot be measured by dependency count alone.

Some services with broad downstream reach did not always produce the highest total impact, while certain shared backend services generated larger expected disruption because of recovery duration and repeated architectural reliance.

This suggests that simple structural rankings are insufficient. Reliability metrics materially change prioritisation decisions.

Another finding was that reducing propagation probability created measurable resilience gains. In practice, this supports engineering investment in loose coupling, fallback mechanisms, and dependency isolation patterns. Some of these conditions are also consistent with recognised microservice bad smell literature, where excessive coupling increases fragility (Taibi and Lenarduzzi, 2018).

5.12 Chapter Summary

This chapter demonstrated that the implemented analytical framework successfully generated meaningful service-level and system-level reliability insights across multiple scenarios. Higher failure rates and slower recovery times increased vulnerability, while reduced propagation lowered cascading risk. Shared backend platforms repeatedly emerged as critical components, and the model consistently distinguished lower-risk from higher-risk services.

The project therefore met its core objectives of modelling cascading failure impact, comparing vulnerability conditions, and identifying critical services through an interpretable graph-based approach. The following chapter critically evaluates the wider significance, limitations, and future development of the study.

6 Evaluation and Conclusion

This chapter critically evaluates the overall success of the project by reviewing the technical implementation, research contribution, project management process, achieved objectives, limitations, and future development opportunities. It draws together findings from earlier chapters to provide a balanced assessment of the practical and academic value of the study.

6.1 Final Evaluation

The primary aim of this project was to determine whether graph-based analytical modelling could provide a practical and interpretable method for quantifying cascading failure impact and system risk in cloud microservice architectures. The findings presented in Chapter 5 indicate that this aim was substantially achieved.

The developed framework transformed a cloud-style microservice environment into a directed dependency graph and incorporated operational reliability variables including failure rate, propagation probability, and Mean Time to Recovery (MTTR). This enabled estimation of service-level and system-level risk without requiring access to live production systems or confidential enterprise telemetry.

The outputs demonstrated meaningful variation across tested scenarios. Reduced propagation probability lowered overall vulnerability, while increased MTTR and higher failure rates significantly increased total risk exposure. These outcomes are logically consistent with real operational behaviour, suggesting that the model captured important reliability dynamics.

At service level, shared data-layer and infrastructure components repeatedly emerged as high-impact dependencies. This indicates that common platform services may create greater systemic business risk than some visible user-facing applications.

Overall, the project met its objectives and produced a functioning analytical framework with measurable outputs, interpretable metrics, and practical relevance.

6.2 Achievement of Objectives

The project objectives were substantially fulfilled. The objective of representing a microservice system as a dependency graph was achieved through the creation of a node-edge architectural model in which services and dependencies were explicitly encoded.

The objective of incorporating realistic reliability parameters was met through the integration of failure rate, propagation probability, and MTTR within the analytical calculations.

The objective of estimating cascading failure impact was achieved through outputs including Expected System Impact, Dependent Service Loss, and Affected Service Ratio.

The objective of comparing vulnerability across scenarios was satisfied through five controlled experiments and comparative System Vulnerability Index (SVI) analysis.

Finally, the objective of identifying critical services requiring resilience prioritisation was achieved through service rankings and the repeated identification of recurring high-impact components across multiple scenarios.

These outcomes indicate a strong level of objective completion.

6.3 Evaluation of Research Question

The central research question asked whether graph-based analytical modelling, using dependency graphs and scenario-based output metrics, can provide a practical and interpretable method for quantifying cascading failure impact and system risk in cloud microservice architectures.

Based on the findings of this study, the results suggest that the answer is positive within the scope of the model assumptions, selected architecture, and available data. The developed framework successfully identified higher-risk services, measured comparative system vulnerability across multiple scenarios, and produced transparent outputs that can be readily interpreted by engineers and decision-makers.

The scenario analysis demonstrated that changes in failure rate, recovery time, and propagation probability materially influenced overall system risk. In addition, recurring critical services were consistently identified across different operating conditions, indicating that the model captured meaningful structural dependencies rather than producing arbitrary rankings.

However, the framework should be interpreted as a decision-support tool rather than an exact predictor of real production incidents. The study relied on synthetic parameters and simplified assumptions, including static probabilities and strongest-path approximation. Despite these limitations, the model proved effective for comparative risk assessment and resilience prioritisation.

Overall, the research question was supported, demonstrating that graph-based analytical modelling can provide a useful, practical, and interpretable approach for early-stage resilience assessment in distributed cloud systems.

6.4 Project Management Reflection

The project was delivered through a staged development approach consisting of background research, architecture selection, Java implementation, scenario design, result generation, and final

report preparation. This phased structure supported manageable progress and reduced the risk of late-stage implementation failure.

Some adjustments were required during development. The original Monte Carlo simulation approach was useful conceptually, but the later analytical impact model proved more suitable for faster scenario comparison, repeatable outputs, and clearer metrics. This change improved delivery efficiency while strengthening alignment with the research objectives.

Time management was effective overall, although report writing, editing, and chapter refinement required more time than initially expected. This reflects the common challenge that documentation and critical reflection often demand substantial effort beyond technical implementation alone.

6.5 Insights Gained

Several important technical, research, and managerial insights were gained throughout the project.

From a technical perspective, the study demonstrated that dependency structure strongly influences failure impact. Highly connected services are not always the most dangerous, as recovery time and failure likelihood can materially change prioritisation. The findings also showed that shared infrastructure may dominate total system risk, and that relatively simple graph models can still provide meaningful operational intelligence.

From a research perspective, the project reinforced that interpretable models may offer stronger practical value than unnecessarily complex methods when decision support is the goal. Scenario analysis also proved useful where real production data was unavailable.

From a managerial perspective, iterative development reduced delivery risk, while early simplification decisions improved final quality. The project also highlighted the importance of clear visual presentation when communicating technical findings.

6.6 Comparison to Literature

The literature review identified three recurring themes: microservice resilience mechanisms, chaos engineering and runtime testing, and structural dependency modelling. Existing studies often address these themes separately. For example, resilience literature commonly focuses on retries, replication, and circuit breakers, while graph-based studies focus on topology and propagation pathways, and chaos engineering focuses on controlled fault injection.

This project contributes by combining structural dependency analysis with operational reliability parameters in one lightweight framework. The findings align with prior research showing that cascading effects emerge from interconnected systems rather than isolated component failures.

However, the study extends these ideas into a cloud microservice context through service-level rankings, scenario-based vulnerability metrics, and practical decision-support outputs. In this respect, the project responds directly to the gap identified in Chapter 3 regarding low-cost proactive vulnerability assessment.

6.7 Reflection on Challenges

Two principal challenges were encountered during the project.

The first concerned **data availability**. Access to real cloud incident datasets containing complete service dependency mappings, failure frequencies, propagation behaviour, and recovery durations is limited because such information is commercially sensitive, security-relevant, or incompletely disclosed. Consequently, this study relied on synthetic but realistic parameter values informed by published incidents, industry practice, and recognised distributed system designs.

The second challenge concerned **model evolution**. An earlier stochastic FailureEngine prototype used Monte Carlo-style simulation to trigger failures and propagate disruption probabilistically through dependencies. While useful for exploring cascading behaviour conceptually, repeated runs produced variable outputs and required multiple iterations for stable comparison.

The final analytical ImpactCalculator model proved more suitable for the research objectives. It generated deterministic outputs, clearer service rankings, faster scenario comparison, and more interpretable metrics such as Expected System Impact and SVI. This transition improved both practicality and research quality.

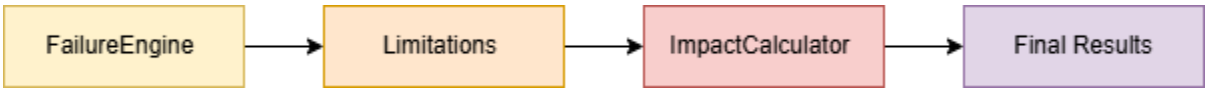

Figure 10 Evolution from stochastic prototype to final analytical model. Source: Author

6.8 Limitations

Despite positive outcomes, the project has several limitations. The model relied on synthetic rather than enterprise operational data, meaning parameter realism cannot be guaranteed. Propagation probabilities were static rather than time-varying, and the strongest-path simplification was used instead of full multi-path union probability modelling.

The framework also did not incorporate live telemetry, user traffic demand patterns, or multiple industry architecture case studies. As a result, findings should be interpreted as indicative comparative insights rather than absolute operational predictions.

6.9 Commercial and Practical Relevance

The project has meaningful practical relevance because organisations increasingly depend on cloud services where outages may generate lost revenue, SLA penalties, reputational damage, customer churn, and emergency engineering costs.

A lightweight risk-scoring framework of this type could support resilience investment prioritisation, migration planning, dependency reviews, business continuity preparation, and architecture governance. Because the model uses configurable inputs and modest tooling requirements, it could potentially be adopted at lower cost than large-scale resilience programmes or extensive live experimentation.

6.10 Future Work

Several developments could strengthen future versions of the framework. Real incident and monitoring data could be used to calibrate realistic failure rates, propagation probabilities, and recovery times. The strongest-path assumption could be replaced with a multi-path union probability model to better represent redundancy and alternative routes.

Further work could also introduce dynamic probabilities that vary according to traffic load, deployment windows, or incident severity. A real-time dashboard integrating observability data could enable continuous risk scoring. Additional studies might compare AWS, Azure, and Google Cloud style architectures, while an economic layer could convert technical risk metrics into direct financial impact estimates.

6.11 Overall Conclusion

This dissertation investigated whether graph-based analytical modelling could be used to quantify cascading failure impact and system risk in cloud microservice architectures.

The study successfully developed a directed dependency framework integrating failure rate, propagation probability, and Mean Time to Recovery (MTTR) into measurable outputs including Expected System Impact and the System Vulnerability Index (SVI). Scenario-based experimentation showed that vulnerability changed significantly under different operating conditions.

The results demonstrated that increased MTTR and higher failure rates strongly amplified system risk, while reduced propagation probability lowered cascading impact. Shared infrastructure and data-layer services generated disproportionate disruption, with Cloud SQL repeatedly emerging as the highest-risk component across multiple scenarios.

Academically, the project contributes a structured method for combining dependency topology with operational reliability metrics in a lightweight analytical framework. Practically, it offers organisations a feasible way to identify critical dependencies and prioritise resilience improvements before incidents occur.

Although the model does not claim exact prediction of real-world outages, it proved effective as a comparative and interpretable decision-support approach. The use of synthetic parameters and simplifying assumptions limits direct real-world generalisation, but does not reduce the value of the framework for controlled analysis.

Therefore, the findings support the original research question, indicating that graph-based analytical modelling can provide a practical and interpretable method for assessing cascading failure risk in distributed cloud systems. The project provides a strong foundation for future work involving real operational data, dynamic risk modelling, and live resilience monitoring.

7 References

Avižienis, A., Laprie, J.C., Randell, B. and Landwehr, C. (2004) 'Basic concepts and taxonomy of dependable and secure computing', *IEEE Transactions on Dependable and Secure Computing*, 1(1), pp. 11–33.

Donne Martin (n.d.) System Design Primer – Twitter architecture. Available at: https://github.com/donnemartin/system-design-primer (Accessed: 11 March 2026).

Google Cloud (2025) Google Cloud status dashboard and incident history. Available at: https://status.cloud.google.com/ (Accessed: 11 March 2026).

Mailewa, A.B., Akuthota, A. and Mohottalalage, T.M.D. (2025) 'A review of resilience testing in microservices architectures: implementing chaos engineering for fault tolerance and system reliability', in *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, pp. 236–242. doi:10.1109/CCWC62904.2025.10903891.

O'Halloran, J., Murphy, P. and Lee, S. (2021) 'A graph theory approach to predicting functional failure propagation during conceptual systems design, 9(3), pp. 88–104.

Taibi, D. and Lenarduzzi, V. (2018) 'On the definition of microservice bad smells', *IEEE Software*, 35(3), pp. 56–62.

Velepucha, J. and Flores, M. (2023) 'A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges', *International Journal of Software Architecture*, 18(1), pp. 15–29.

8 Appendix

Appendix A - Java Implementation Evidence

```java
public List<ImpactResult> calculateAllImpacts() {  1 usage   pabis21
    List<ImpactResult> results = new ArrayList<>();

    for (Service s : graph.services) {
        results.add(calculateImpact(s));
    }

    return results;
}

public double calculateSVI(List<ImpactResult> results) {  1 usage   pabis21
    double sum = 0.0;
    for (ImpactResult r : results) {
        sum += r.expectedSystemImpact;
    }
    return sum;
}

public ImpactResult getMaxRiskService(List<ImpactResult> results) {  1 usage   pabis21
    ImpactResult max = results.get(0);

    for (ImpactResult r : results) {
        if (r.expectedSystemImpact > max.expectedSystemImpact) {
            max = r;
        }
    }

    return max;
}
```

Figure 11 Core Java methods used to calculate service impact, System Vulnerability Index (SVI), and highest-risk service ranking.

Appendix B – Input Dataset Configuration

Figure 12 CSV configuration files used to define baseline and scenario-specific service/dependency parameters.

```csv
source,target,probability
cloud_cdn,google_cloud_dns,0.95
cloud_load_balancing,cloud_cdn,0.95
frontend_cloud_run,cloud_load_balancing,0.95
accounts_api_cloud_run,frontend_cloud_run,0.90
read_api_cloud_run,frontend_cloud_run,0.90
category_service_cloud_run,accounts_api_cloud_run,0.85
budget_service_cloud_run,accounts_api_cloud_run,0.85
transaction_extraction_service_cloud_run,accounts_api_cloud_run,0.80
notification_service_cloud_run,accounts_api_cloud_run,0.75
category_service_cloud_run,read_api_cloud_run,0.80
budget_service_cloud_run,read_api_cloud_run,0.80
accounts_api_cloud_run,cloud_sql,0.90
transaction_extraction_service_cloud_run,cloud_sql,0.85
read_api_cloud_run,cloud_sql_read_replica,0.90
accounts_api_cloud_run,cloud_memorystore,0.85
read_api_cloud_run,cloud_memorystore,0.85
notification_service_cloud_run,queue_pub_sub,0.80
transaction_extraction_service_cloud_run,queue_pub_sub,0.85
transaction_extraction_service_cloud_run,cloud_storage,0.85
```

Figure 13 Example dependency relationships and propagation probabilities used by the model.

```
service,failure_rate,mttr
google_cloud_dns,0.01,5
cloud_cdn,0.01,5
cloud_load_balancing,0.02,5
frontend_cloud_run,0.03,10
accounts_api_cloud_run,0.04,15
read_api_cloud_run,0.04,15
category_service_cloud_run,0.05,20
budget_service_cloud_run,0.05,20
notification_service_cloud_run,0.05,20
transaction_extraction_service_cloud_run,0.06,25
cloud_memorystore,0.03,10
queue_pub_sub,0.03,10
cloud_sql,0.07,30
cloud_sql_read_replica,0.05,25
cloud_storage,0.04,20
```

Figure 14 Example service reliability inputs including failure rate and MTTR.

Appendix C – Output Evidence

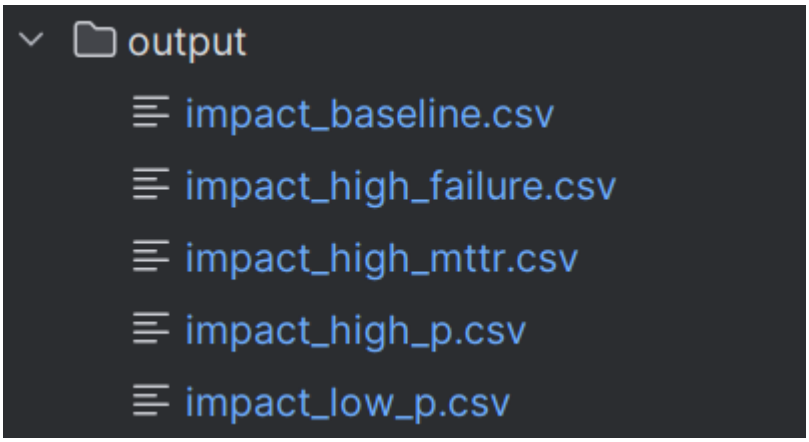

Figure 15 Generated scenario output files exported by the Java implementation.

```
service,dependent_service_loss,self_downtime,expected_system_impact,affected_service_ratio
google_cloud_dns,94.22836249999999,5.0,0.942283625,0.6428571428571429,9
cloud_cdn,94.18775000000001,5.0,0.9418775000000001,0.5714285714285714,8
cloud_load_balancing,94.145,5.0,1.8829,0.5,7
frontend_cloud_run,89.10000000000001,10.0,2.673,0.42857142857142855,6
accounts_api_cloud_run,69.0,15.0,2.7600000000000002,0.2857142857142857,4
read_api_cloud_run,32.0,15.0,1.28,0.14285714285714285,2
category_service_cloud_run,0.0,20.0,0.0,0.0,0
budget_service_cloud_run,0.0,20.0,0.0,0.0,0
notification_service_cloud_run,0.0,20.0,0.0,0.0,0
transaction_extraction_service_cloud_run,0.0,25.0,0.0,0.0,0
cloud_memorystore,84.15,10.0,2.5245,0.42857142857142855,6
queue_pub_sub,37.25,10.0,1.1175,0.14285714285714285,2
cloud_sql,78.85,30.0,5.5195,0.35714285714285715,5
cloud_sql_read_replica,42.300000000000004,25.0,2.115,0.21428571428571427,3
cloud_storage,21.25,20.0,0.85,0.07142857142857142,1
```

Figure 16 Example analytical output containing dependent loss, downtime, expected impact, and affected ratio.

Appendix D – R Analysis Scripts

```r
# ------------------------------
# Reorder services by baseline expected impact
# ------------------------------
service_order <- baseline %>%
  arrange(desc(expected_system_impact)) %>%
  pull(service)

all_data$service <- factor(all_data$service, levels = service_order)
baseline$service <- factor(baseline$service, levels = service_order)

# ------------------------------
# Calculate SVI per scenario
# ------------------------------
svi_data <- all_data %>%
  group_by(scenario) %>%
  summarise(SVI = sum(expected_system_impact, na.rm = TRUE), .groups = "drop")

cat("System Vulnerability Index (SVI) by scenario:\n")
print(svi_data)

# ------------------------------
# Find max-risk service per scenario
# ------------------------------
max_risk <- all_data %>%
  group_by(scenario) %>%
  slice_max(order_by = expected_system_impact, n = 1, with_ties = FALSE) %>%
  ungroup()

cat("Max-risk service by scenario:\n")
print(max_risk)

# ------------------------------
# Graph 1: SVI comparison
# ------------------------------
p1 <- ggplot(svi_data, aes(x = scenario, y = SVI, fill = scenario)) +
  geom_col() +
  labs(
    title = "System Vulnerability Index Across Scenarios",
    x = "Scenario",
    y = "SVI"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "right")

print(p1)
ggsave("output/svi_comparison.png", plot = p1, width = 8, height = 5)
```

Figure 17 R script used to aggregate scenario results and generate comparative charts.

Appendix E – Source Code Repository

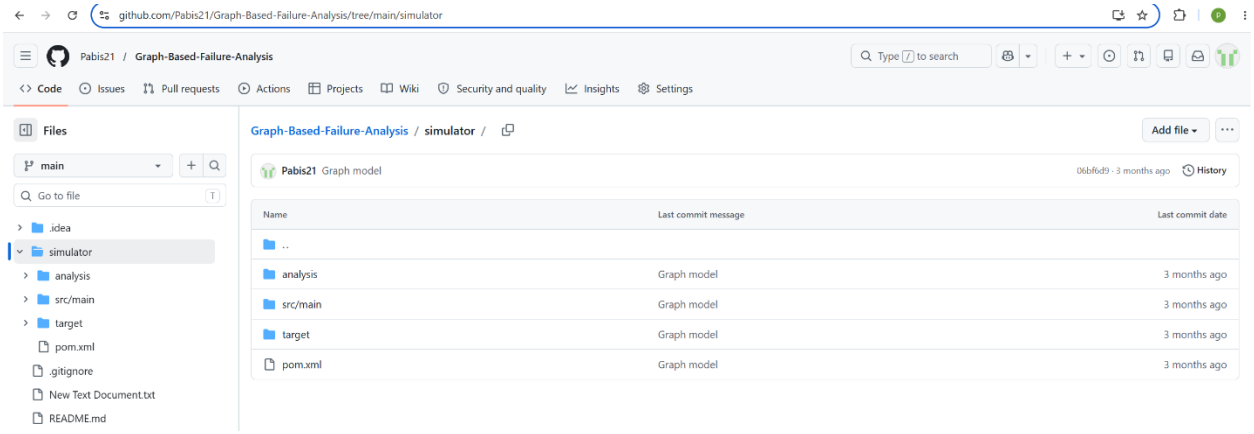

Figure 18 Source code repository: https://github.com/Pabis21//Graph-Based-Failure-Analysis